



Εισαγωγή στον Προγραμματισμό

Εργασία #0

Οκτώβριος 2026

Στόχος της εργασίας είναι να αποκτήσουμε εμπειρία με τα βασικά εργαλεία του προγραμματιστή και να γράψουμε ένα από τα πρώτα προγράμματα σε C. Συγκεκριμένα, στοχεύουμε γνωριμία με τα ακόλουθα:

1. git, version control, repositories, GitHub.
2. Γραμμή εντολών Linux και bash.
3. Χρήση ακεραίων στην C.
4. Χρήση βασικών δομών (loops, if, functions).
5. Δεδομένα εισόδου από πρότυπη είσοδο.

Υποβολή Εργασίας. Όλες οι υποβολές των εργασιών θα γίνουν μέσω GitHub και συγκεκριμένα στο github.com/progintro [5]. Προκειμένου να ξεκινήσεις, μπορείς να δεχτείς την άσκηση με αυτήν την: πρόσκληση [6].

Σημαντικό: Στην εργασία, όπως και σε όλες τις εργασίες του μαθήματος, δεν επιτρέπεται η χρήση τεχνητής νοημοσύνης για να ολοκληρώσετε τις ασκήσεις. Οποιαδήποτε χρήση θα οδηγήσει σε μηδενισμό στο μάθημα.

1. Νέο URL στο GitHub (pages - 30 Μονάδες)

Έχεις λογαριασμό στο GitHub; Έχεις ξαναφτιάξει repository; Έχεις φτιάξει URL στο διαδίκτυο; Μετά από αυτήν την άσκηση θα μπορείς να απαντήσεις "ναι" σε όλα. Αν ήδη μπορείς να απαντήσεις ναι, η άσκηση αυτή (ελπίζουμε) να μην σου πάρει πολύ χρόνο.

Η ιστοσελίδα του μαθήματος βασίζεται σε ένα public GitHub repository [4]. Σκοπός αυτής της άσκησης είναι να δημιουργήσεις ένα καινούριο repository στον προσωπικό σου λογαριασμό GitHub και να το συνδέσεις με GitHub Pages προκειμένου να είναι διαθέσιμο στο διαδίκτυο. Αυτό το tutorial [7] μπορεί να σε βοηθήσει.

Τεχνικές Προδιαγραφές

Η υποβολή σου, πρέπει να έχει τα ακόλουθα χαρακτηριστικά:

- Repository Name: progintro/hw0-2026-<YourUsername>
- Αρχείο Λύσης Filepath: pages/solution.txt
- README Filepath (optional): pages/README.md

Έστω ότι το GitHub username σου είναι YourGitHubUsername. Το αρχείο solution.txt πρέπει να περιέχει το URL με το domain YourGitHubUsername.github.io που δημιουργήσατε. Αν χρησιμοποιήσετε κάποιο *.github.io URL που δεν δημιουργήσατε η άσκηση μηδενίζεται. Ενδεικτική συμπεριφορά για μια επιτυχημένη υποβολή (**προσοχή: η γραμμή με την εντολή curl εκτείνεται σε δύο γραμμές, πρέπει να αντιγράψετε και τις δύο προκειμένου να τρέξετε αυτήν την εντολή¹**):

```
# Check out solution URL
$ cat solution.txt
YourUsername.github.io
# Ensure the URL exists
$ curl --output /dev/null --silent --head --fail YourGitHubUsername.github.io \
  && echo "URL exists" || echo "URL does not exist"
URL exists
```

Το αρχείο README.md είναι προαιρετικό αν θέλετε να προσθέσετε κάτι στην υποβολή σας.

2. Παιχνίδια με Κονσόλα (cmdline - 170 Μονάδες)

Έχεις χρησιμοποιήσει την γραμμή εντολών; Linux; Γνωρίζεις πως να κάνεις SSH σε απομακρυσμένους servers και να λύνεις προβλήματα με εντολές κονσόλας; Αν όχι, καιρός να μάθουμε! Αν ναι, τότε αυτή η άσκηση θα σου είναι παιχνιδάκι.

Σε αυτήν την άσκηση θα μπούμε με ssh σε έναν απομακρυσμένο server που βρίσκεται στην IP διεύθυνση 35.169.96.19 μέσω τερματικού και θα ανακτήσουμε συγκεκριμένες πληροφορίες. Υπάρχουν διαφορετικοί χρήστες (byte0, byte1, ...) που αντιστοιχούν σε διαφορετικά προβλήματα/υποερωτήματα. Για όλους τους χρήστες ο κωδικός για να μπείτε είναι bits&bytes . Σε κάθε ένα υποερώτημα θα αναζητούμε ένα string που μπορεί να είναι μια μόνο λέξη ή πολλές λέξεις διαχωρισμένες με τον χαρακτήρα underscore _ - για παράδειγμα: something_like_that.

¹Προσοχή στο backslash \ στην γραμμή εντολών, αυτό που θέλουμε να δείξουμε είναι ότι η εντολή είναι μία αλλά εκτείνεται σε δύο γραμμές.

byte0: Αλλάζοντας Φάκελο (cd)

Ο χρήστης byte0 είναι ελαφρώς μανιακός με την οργάνωση. Του αρέσει να οργανώνει όλα του τα αρχεία σε φακέλους, υποφακέλους κ.ο.κ.. Με αυτά και με αυτά όμως, δεν μπορεί να βρει το αρχείο του, μπορείτε να τον βοηθήσετε; Για να μπειτε στον server τρέξτε:

```
ssh byte0@35.169.96.19
```

και βάλτε τον κωδικό που γράψαμε παραπάνω. Η εντολή cd μπορεί να σας φανεί χρήσιμη όπως και η εντολή cat (ίσως και το tab / autocomplete :)). Όταν διαβάσετε τα περιεχόμενα του αρχείου, σημειώστε το string που βρήκατε μέσα στο αρχείο ώστε να το προσθέσετε στο solution.txt (δες παρακάτω στις τεχνικές προδιαγραφές).

byte1: Μια Εντολή Άγνωστη

Ο χρήστης byte1 έφτιαξε ένα καινούριο πρόγραμμα με άγνωστη σε μας λειτουργία ονόματι supercalifragilisticexpialidocious. Πως μπορείτε να βρείτε τι κάνει;

byte2: Τα Άπαντα του Shakespeare (grep)

Ο χρήστης byte2 έχει δύο βασικά χαρακτηριστικά: (1) είναι μεγάλος θαυμαστής του Shakespeare και (2) είναι κάπως μυστικοπαθής. Αποφάσισε να συνδυάσει τα δύο και καταχώνιασε ένα μυστικό string μέσα στην συλλογή του με τα άπαντα του Shakespeare. Μπορείτε να τον βοηθήσετε να το βρεί;

Το μόνο που γνωρίζουμε είναι πως το κρυφό κείμενο περιέχει το string "will find". Η εντολή grep μπορεί να σας φανεί χρήσιμη.

byte3: Τα Άπαντα του Shakespeare Άλλαξαν (diff)

Ο χρήστης byte3 αποφάσισε να αλλάξει κάτι στα άπαντα του Shakespeare ελπίζοντας πως η αλλαγή θα περάσει απαρατήρητη. Ελέγξαμε τον αριθμό των λέξεων και χαρακτήρων του κάθε αρχείου:

```
byte3@ip-172-31-37-131:~$ wc -w *.txt
901325  shakespeare.modified.txt
901325  shakespeare.txt
1802650 total
byte3@ip-172-31-37-131:~$ wc -c *.txt
5458203 shakespeare.modified.txt
5458199 shakespeare.txt
```

και παρατηρήσαμε πως ενώ ο αριθμός λέξεων παρέμεινε ίδιος (901.325), ο αριθμός των χαρακτήρων άλλαξε. Επομένως, υποψιαζόμαστε πως άλλαξε μια λέξη του αυθεντικού με μια καινούρια. Βρείτε την καινούρια λέξη—η εντολή diff ίσως σας φανεί χρήσιμη.

byte4: Λαβύρινθος (find)

Ο χρήστης byte4 είναι συγγενής του byte2 παραπάνω και ελαφρώς πιο μυστικοπαθής. Προκειμένου να "ανεβάσει" το παιχνίδι του αποφάσισε να κρύψει τα δεδομένα του σε ένα δαιδαλώδες κατασκεύασμα από φακέλους. Μπορείτε και πάλι να τον βοηθήσετε να βρει το μυστικό που έχει κρύψει μέσα σε ένα από όλα τα αρχεία των υποφακέλων; Το μόνο που γνωρίζουμε είναι πως έχει κρύψει το μυστικό του σε ένα αρχείο που λέγεται cup.txt. Η εντολή find μπορεί να σας φανεί χρήσιμη.

byte5: Compile and Run (gcc, mkdir, cp)

Ο byte5 έγραψε ένα προγραμματάκι σε C ονόματι byte5.c. Μεταγλωττίστε το και τρέξτε το προκειμένου να πάρετε τον κωδικό σας! Προσοχή: ο default φάκελος του byte5 δεν σας επιτρέπει να χρησιμοποιήσετε αρχεία, ίσως χρειαστεί να φτιάξετε έναν καινούριο υποφάκελο μέσα στον φάκελο /tmp προκειμένου να τοποθετήσετε το εκτελέσιμό σας. Οι εντολές mkdir, cp, gcc μπορεί να σας φανούν χρήσιμες.

byte6: Αποσυμπίεση αρχείου 1 (unzip)

Κάποιες φορές προκειμένου να μικρύνουμε τον χώρο που πιάνει ένα αρχείο το συμπιέζουμε. Ο χρήστης byte6 έχει ένα αρχείο byte6.zip το οποίο χρειάζεται αποσυμπίεση. Ανοίξτε το για να αποκτήσετε πρόσβαση στα περιεχόμενά του. Το πρόγραμμα unzip μπορεί να σας βοηθήσει.

byte7: Αποσυμπίεση αρχείου 2 (tar)

Υπάρχουν διάφοροι τρόποι και εργαλεία για να συμπίεσεις αρχεία και τα περιεχόμενά τους. Ο χρήστης byte7 χρησιμοποίησε ένα πρόγραμμα που λέγεται tar - μπορείτε να χρησιμοποιήσετε το ίδιο προκειμένου να βρείτε τα περιεχόμενα του byte7.tar.gz;

byte8: Carriage Return (xxd, pico, vim)

Ο χρήστης byte8 μας είπε πως ο κωδικός που ψάχνουμε είναι μέσα στο αρχείο carriage_return.txt. Όμως δοκιμάσαμε να το τυπώσουμε:

```
byte8@ip-172-31-37-131:~$ ls
carriage_return.txt
byte8@ip-172-31-37-131:~$ cat carriage_return.txt
There is absolutely nothing to see here. Move along.
```

και δεν είδαμε κάτι. Μπορείς να μας βοηθήσεις; Πιστεύουμε πως οι εντολές xxd, pico ή vim μπορεί να φανούν χρήσιμες.

byte9: Ένα Περίεργο Όνομα (cat)

Ο χρήστης byte9 ανέφερε πως ο κωδικός του είναι μέσα στον φάκελο /home/byte9 - μπορείτε να μας βοηθήσετε να τον ανακτήσουμε; Ίσως σε βοηθήσει να θυμηθείς τι είναι το filepath (μονοπάτι) ενός αρχείου.

byte10: Το 67ο Όνομα (sort, head)

Ο χρήστης byte10 έχει ένα αρχείο γεμάτο με ονόματα και θέλει να τα ταξινομήσει αλφαβητικά και να βρει το 67ο όνομα στην σειρά. Γνωρίζει ότι το 1ο όνομα είναι το Aarika - μπορείτε να τον βοηθήσετε να βρει το 67ο; Οι εντολές sort και head μπορούν να συνδυαστούν και να βοηθήσουν.

byte11: Επιλογές Ονόματος (sort, uniq)

Ο χρήστης byte11 μόλις απέκτησε ένα παιδάκι και προκειμένου να μην το κοροϊδεύουν στο σχολείο αποφάσισε να του δώσει το πιο συχνό όνομα εκείνης της χρονιάς. Στο αρχείο births.txt περιέχονται τα ονόματα όλων των φετινών γεννήσεων. Μπορείτε να τον βοηθήσετε να βρει το πιο συχνό όνομα; Οι εντολές sort και uniq ίσως σας φανούν χρήσιμες.

byte12: Βρες το Μονοπάτι (which, PATH)

Ο byte12 είναι κάπως αφηρημένος. Μόλις έφτιαξε το πρόγραμμα tuxsay αλλά δεν θυμάται που το τοποθέτησε και δεν πρόλαβε να φτιάξει ένα manual page. Παρόλα αυτά, μπορεί να το τρέξει κανονικά. Μπορείτε να βρείτε το basename του φακέλου μέσα στον οποίο βρίσκεται το πρόγραμμα tuxsay;

byte13: Commit Messages (git log)

Ο byte13 μόλις πήρε πρόσβαση σε ένα git repository (αποθετήριο) και θέλει να δει τα commit messages (μηνύματα για προηγούμενες αλλαγές) που έκαναν οι χρήστες που συνεισέφεραν στο repository. Μπορείτε να διαβάσετε αυτά τα μηνύματα;

byte14: Ιστορικό (git log -p)

Ο byte14 ξεκίνησε να γράφει μια νουβέλα με την βοήθεια του ΑΙ. Μετά από αρκετές αλλαγές διαπίστωσε ότι το ΑΙ είχε προσθέσει (κατά λάθος;) έναν από τους κωδικούς του μέσα στο κείμενο. Ευτυχώς το είδε, το αφαίρεσε, έκανε commit τις αλλαγές και πλέον ξέρει ότι δεν είναι ορατό σε άλλους χρήστες. Έχει δίκιο ο byte14;

byte15: Κωδικός Εξόδου

Ο byte15 έφτιαξε ένα προγραμματάκι και το "έκρυψε" στον τρέχοντα φάκελό του. Μπορείτε να τον βοηθήσετε να το βρει; Μήπως η εντολή `ls` έχει κάποιο όρισμα που βοηθάει; Πως τυπώνουμε τον κωδικό εξόδου (exit code) ενός προγράμματος;

byte16: Θορυβώδες Σήμα

Ο byte16 μόλις είδε την ταινία *A Beautiful Mind* και αποφάσισε να κρύψει τα μυστικά του μέσα σε θόρυβο. Δοκιμάσαμε να τρέξουμε το πρόγραμμα πολλές φορές αλλά ειλικρινά δεν μπορούμε να βρούμε το μυστικό μήνυμα. Το μόνο που γνωρίζουμε είναι πως τυπώνει το μυστικό μήνυμα στο `stdout` (πρότυπη έξοδο) ενώ τον θόρυβο στο `stderr` (πρότυπη έξοδο σφάλματος). Μήπως μπορείτε να μας βοηθήσετε;

byte17: Open the Vault (bonus / προαιρετικό)

Ο byte17 είναι χρήστης του vault, ενός υπερασφαλούς προγράμματος που εμφανίζει τα περιεχόμενα του λογαριασμού σου, μόνο εφόσον γνωρίζεις το μυστικό PIN. Δυστυχώς, ο byte17 ξέχασε το PIN του! Μπορείτε να τον βοηθήσετε να ανακτήσει τα περιεχόμενα του λογαριασμού του;

Τεχνικές Προδιαγραφές

Η υποβολή σου, πρέπει να έχει τα ακόλουθα χαρακτηριστικά:

- Repository Name: `progintro/hw0-2026-<YourUsername>`
- README Filepath: `cmdline/README.md`
- Αρχείο Λύσης Filepath: `cmdline/solution.txt`
- Κάθε γραμμή που θα προστεθεί στο αρχείο `solution.txt` πρέπει να γίνει με διαφορετικό `git commit`. Κοινώς, θέλουμε να δούμε τουλάχιστον *17* διαφορετικά `git commits`. Η κάθε γραμμή πρέπει να προστεθεί όταν λύσετε το αντίστοιχο πρόβλημα, όχι όλες μαζί στο τέλος.

Το περιεχόμενο του `solution.txt` πρέπει να έχει την ακόλουθη μορφή:

```
$ cat solution.txt
byte0: ...
byte1: ...
byte2: ...
...
```

Όπου οι τελείες (...) περιγράφουν το μέρος της λύσης που έχει παραληφθεί.

Στο αρχείο README.md πρέπει να γράψετε μια σύντομη περιγραφή για τον τρόπο που λύσατε το κάθε πρόβλημα.

Αν σου άρεσε αυτή η άσκηση ή έχεις κολλήσει με ένα από τα παραπάνω και δεν ξέρεις πως να συνεχίσεις, υπάρχουν και άλλα site online στα οποία μπορείς να εξασκηθείς όπως για παράδειγμα το overthewire [8] τα οποία μπορούν να σου δώσουν ιδέες. Σημείωση: μέχρι το επίπεδο 11 τα προβλήματα σε αυτό το site είναι εντός θέματος για το μάθημα, από εκεί και πέρα γίνονται πιο δύσκολα / απαιτούν γνώσεις που θα αποκτήσουμε σε μεγαλύτερα έτη.

3. Η Εικασία του Goldbach (100 Μονάδες)

Το 1742, ο μαθηματικός Christian Goldbach έστειλε μια επιστολή προς τον Leonhard Euler με μια παρατήρηση που έκανε σχετικά με τους πρώτους αριθμούς. Συγκεκριμένα, ο Christian παρατήρησε πως:

Κάθε άρτιος ακέραιος μεγαλύτερος του 2 μπορεί να γραφτεί ως άθροισμα δύο πρώτων αριθμών.

Υπενθυμίζουμε πως ένας θετικός ακέραιος μεγαλύτερος του 1 λέγεται *πρώτος* όταν οι μόνοι θετικοί διαιρέτες του είναι το 1 και ο ίδιος. Για παράδειγμα, οι πρώτοι αριθμοί ξεκινούν ως²:

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, ...

Παίρνοντας την παραπάνω παρατήρηση και βάζοντας λίγο χρόνο, φαίνεται να ισχύει για όλους άρτιους δοκιμάσαμε:

$$4 = 2 + 2$$

$$6 = 3 + 3$$

$$8 = 3 + 5$$

$$10 = 3 + 7$$

$$20 = 3 + 17$$

$$42 = 5 + 37$$

Ισχύει όμως για όλους τους άρτιους αριθμούς; Αυτή η σχετικά απλή ερώτηση πήρε το όνομα *Εικασία του Goldbach* [1] και έμελε να θεωρηθεί ένα από τα πιο δύσκολα μαθηματικά προβλήματα όλων των εποχών. Τους τελευταίους τρεις αιώνες, μαθηματικοί ανά την υφήλιο προσπάθησαν να την αποδείξουν (ή να αποδείξουν πως δεν ισχύει!) [3] μέχρι στιγμής ανεπιτυχώς (ανάμεσά τους Euler, Hardy και άλλοι). Με

²Ο αριθμός 1 δεν θεωρείται πρώτος.

την βοήθεια των υπολογιστών και του προγραμματισμού, γνωρίζουμε σήμερα πως η εικασία ισχύει για όλους τους άρτιους μέχρι το $4 \cdot 10^{18}$.

Στη συγκεκριμένη άσκηση δεν θα σας ζητήσουμε—ακόμα—να την αποδείξετε. Θα χρειαστεί όμως να γράψετε ένα πρόγραμμα που μπορεί να ελέγχει αν ένας αριθμός είναι πρώτος και να αναζητά μια αναπαράσταση Goldbach για όσους αριθμούς διαλέξει ο χρήστης. Οι τεχνικές προδιαγραφές της υλοποίησης ακολουθούν.

Τεχνικές Προδιαγραφές

- Repository Name: progintro/hw0-2026-<YourUsername>
- Αρχείο C (Filepath): goldbach/src/goldbach.c
- Το αρχείο C που θα υποβληθεί πρέπει να μεταγλωττίζεται χωρίς ειδοποιήσεις για λάθη με την ακόλουθη εντολή σε ένα από τα μηχανήματα του εργαστηρίου:

```
gcc -O0 -m32 -Wall -Wextra -Werror -pedantic -o goldbach goldbach.c
```

- README Filepath: goldbach/README.md
- Το αρχείο goldbach.c δεν πρέπει να ξεπερνά τα 8KB (8192 bytes). Μπορείτε να ελέγξετε το μέγεθός του με την εντολή `wc -c goldbach.c`.
- Το πρόγραμμα πρέπει να διαβάσει επαναλαμβανόμενα από την πρότυπη είσοδο (stdin) έναν ακέραιο και έναν χαρακτήρα επιλογής.
- Το πρόγραμμά σας θα πρέπει να μπορεί να διαχειριστεί αριθμούς στο εύρος:

$$0 \leq n \leq 10.000.000.000.$$

- Μετά την ανάγνωση του αριθμού, το πρόγραμμα πρέπει να διαβάζει έναν χαρακτήρα:
 - p για έλεγχο πρώτου αριθμού,
 - g για έλεγχο Goldbach.
- Ένας άρτιος μπορεί συχνά να γραφτεί ως άθροισμα δύο πρώτων με περισσότερους από έναν τρόπους (π.χ., $10 = 3 + 7 = 5 + 5$). Προκειμένου η απάντηση να είναι μοναδική, για τον έλεγχο Goldbach το πρόγραμμά σας πρέπει να τυπώνει το ζεύγος $n = p + q$ με $p \leq q$ όπου ο p είναι ο **μικρότερος δυνατός** πρώτος. Ο μικρότερος από τους δύο πρώτους (p) τυπώνεται πρώτος. Για παράδειγμα, για το 10 η μόνη αποδεκτή απάντηση είναι $10 = 3 + 7$.

- Αν ζητηθεί έλεγχος Goldbach για αριθμό που δεν είναι άρτιος μεγαλύτερος του 2, το πρόγραμμα τυπώνει το μήνυμα Goldbach's conjecture is applicable only to even numbers greater than 2! και συνεχίζει κανονικά με τον επόμενο αριθμό.
- Αν συναντηθεί EOF κατά την προσπάθεια ανάγνωσης (είτε του αριθμού είτε του χαρακτήρα επιλογής), το πρόγραμμα σας πρέπει να τυπώνει EOF detected, exiting και να τερματίζει με κωδικό εξόδου 0.
- Αν δοθεί είσοδος εκτός προδιαγραφών, το πρόγραμμά σας πρέπει να τυπώνει το αντίστοιχο μήνυμα λάθους στην πρότυπη έξοδο σφάλματος (stderr) και να τερματίζει αμέσως με κωδικό εξόδου (exit code) 1:
 - Αν η είσοδος δεν είναι ακέραιος (π.χ., banana): Invalid input: expected an integer
 - Αν ο αριθμός είναι εκτός του παραπάνω εύρους (π.χ., -5): Number should be in the range [0, 10000000000]
 - Αν ο χαρακτήρας επιλογής δεν είναι p ή g: Invalid choice: expected 'p' or 'g'
- Αν, κατά τύχη, βρείτε κάποιον άρτιο που δεν μπορεί να εκφραστεί ως άθροισμα δύο πρώτων ή στην πορεία αποδείξετε την Εικασία του Goldbach παρακαλώ επικοινωνήστε μαζί μας, εγγυώμαστε πως θα πάρετε 10 εφόσον μοιραστείτε την δόξα [2].
- Για κάθε είσοδο εντός των παραπάνω ορίων, το πρόγραμμα πρέπει να ολοκληρώνει τον αντίστοιχο έλεγχο (p ή g) σε **λιγότερο από 1 δευτερόλεπτο** στα μηχανήματα του εργαστηρίου. Για αναφορά, μια ενδεικτική λύση χρειάζεται μόλις μερικά εκατοστά του δευτερολέπτου ακόμα και για τις πιο "δύσκολες" εισόδους (δείτε το παράδειγμα με την εντολή time παρακάτω).
- Αν κάπου χρησιμοποιήσετε κάποια πηγή, να την προσθέσετε σε σχόλιο μέσα στο αρχείο C.

Παρακάτω παραθέτουμε μια ενδεικτική αλληλεπίδραση με μια σωστή λύση:

```
$ gcc -O0 -m32 -Wall -Wextra -Werror -pedantic -o goldbach goldbach.c
$ ./goldbach
Give me a number: 17
Choose how to test 17 - ('p' for prime, 'g' for goldbach) p
17 is prime
Give me a number: 42
Choose how to test 42 - ('p' for prime, 'g' for goldbach) p
42 is not prime
```

```

Give me a number: 42
Choose how to test 42 - ('p' for prime, 'g' for goldbach) g
42 = 5 + 37
Give me a number: 10
Choose how to test 10 - ('p' for prime, 'g' for goldbach) g
10 = 3 + 7
Give me a number: 4
Choose how to test 4 - ('p' for prime, 'g' for goldbach) g
4 = 2 + 2
Give me a number: 11
Choose how to test 11 - ('p' for prime, 'g' for goldbach) g
Goldbach's conjecture is applicable only to even numbers greater than 2!
Give me a number: 2
Choose how to test 2 - ('p' for prime, 'g' for goldbach) g
Goldbach's conjecture is applicable only to even numbers greater than 2!
Give me a number: 1
Choose how to test 1 - ('p' for prime, 'g' for goldbach) p
1 is not prime
Give me a number: EOF detected, exiting
$ echo $?
0

```

Παραδείγματα με είσοδο εκτός προδιαγραφών:

```

$ ./goldbach
Give me a number: -5
Number should be in the range [0, 10000000000]
$ echo $?
1
$ ./goldbach
Give me a number: banana
Invalid input: expected an integer
$ echo $?
1
$ ./goldbach
Give me a number: 42
Choose how to test 42 - ('p' for prime, 'g' for goldbach) x
Invalid choice: expected 'p' or 'g'
$ echo $?
1

```

Το πρόγραμμα μπορεί φυσικά να χρησιμοποιηθεί και για αρκετά μεγαλύτερες τιμές:

```

$ ./goldbach

```

```
Give me a number: 100000000000
Choose how to test 100000000000 - ('p' for prime, 'g' for goldbach) g
100000000000 = 71 + 9999999929
```

Η είσοδος μπορεί να δοθεί και από αρχείο (ή pipe) αντί για το πληκτρολόγιο, κάτι που είναι χρήσιμο για να μετρήσουμε τον χρόνο εκτέλεσης με την εντολή `time` (εδώ στέλνουμε την έξοδο του προγράμματος σε ένα αρχείο `output.txt`). Οι χρόνοι παρακάτω είναι ενδεικτικοί και θα διαφέρουν από μηχάνημα σε μηχάνημα:

```
$ cat input.txt
9999999967 p
7473202036 g
$ time ./goldbach < input.txt > output.txt

real    0m0.048s
user    0m0.038s
sys      0m0.009s
$ cat output.txt
Give me a number: Choose how to test 9999999967 - ('p' for prime, 'g' for \
goldbach) 9999999967 is prime
Give me a number: Choose how to test 7473202036 - ('p' for prime, 'g' for \
goldbach) 7473202036 = 1877 + 7473200159
Give me a number: EOF detected, exiting
```

Στο αρχείο `README.md` πρέπει να προσθέσετε οποιεσδήποτε παρατηρήσεις σας κατά την διεκπεραίωση της άσκησης. Ο κώδικας απαιτείται να είναι καλά τεκμηριωμένος με σχόλια καθώς αυτό θα είναι μέρος της βαθμολόγησης.

Αναφορές

- [1] Goldbach's Conjecture. https://en.wikipedia.org/wiki/Goldbach%27s_conjecture.
- [2] How to Win a Million Dollars from Faber and Faber. <https://arpita95b.medium.com/how-to-win-a-million-dollars-from-faber-faber-a40431c03086>.
- [3] The Obviously True Theorem No One Can Prove (αν ξεκινήσατε την άσκηση μια μέρα (ή ώρα) πριν την υποβολή συνιστώ να μην πατήσετε το λινκ). <https://www.youtube.com/watch?v=x32Zq-XvID4>.
- [4] DIT. Repository για το μάθημα . <https://github.com/progintro/progintro.github.io>.
- [5] DIT. Οργανισμός για το μάθημα (GitHub progintro) . <https://github.com/progintro>.

- [6] DIT. Πρόσκληση για Εργασία 0 . <https://hw.uoa.gr/courses/progintro/assignments/hw0-2026/accept>.
- [7] Github. GitHub Pages tutorial . <https://docs.github.com/en/pages/quickstart>.
- [8] OverTheWire. Ασκήσεις σε Γραμμή Εντολών . <https://overthewire.org/wargames/bandit/bandit0.html>.